

Solution of Linear Algebraic Equations

2.0 Introduction

The most basic task in linear algebra, and perhaps in all of scientific computing, is to solve for the unknowns in a set of linear algebraic equations. In general, a set of linear algebraic equations looks like this:

$$\begin{aligned}
 a_{00}x_0 + a_{01}x_1 + a_{02}x_2 + \cdots + a_{0,N-1}x_{N-1} &= b_0 \\
 a_{10}x_0 + a_{11}x_1 + a_{12}x_2 + \cdots + a_{1,N-1}x_{N-1} &= b_1 \\
 a_{20}x_0 + a_{21}x_1 + a_{22}x_2 + \cdots + a_{2,N-1}x_{N-1} &= b_2 \\
 \vdots &\vdots \\
 a_{M-1,0}x_0 + a_{M-1,1}x_1 + \cdots + a_{M-1,N-1}x_{N-1} &= b_{M-1}
 \end{aligned} \tag{2.0.1}$$

Here the N unknowns x_j , $j = 0, 1, \dots, N-1$ are related by M equations. The coefficients a_{ij} with $i = 0, 1, \dots, M-1$ and $j = 0, 1, \dots, N-1$ are known numbers, as are the *right-hand side* quantities b_i , $i = 0, 1, \dots, M-1$.

If $N = M$, then there are as many equations as unknowns, and there is a good chance of solving for a unique solution set of x_j 's. Otherwise, if $N \neq M$, things are even more interesting; we'll have more to say about this below.

If we write the coefficients a_{ij} as a matrix, and the right-hand sides b_i as a column vector,

$$\mathbf{A} = \begin{bmatrix} a_{00} & a_{01} & \cdots & a_{0,N-1} \\ a_{10} & a_{11} & \cdots & a_{1,N-1} \\ \vdots & \vdots & \ddots & \vdots \\ a_{M-1,0} & a_{M-1,1} & \cdots & a_{M-1,N-1} \end{bmatrix} \quad \mathbf{b} = \begin{bmatrix} b_0 \\ b_1 \\ \vdots \\ b_{M-1} \end{bmatrix} \tag{2.0.2}$$

then equation (2.0.1) can be written in matrix form as

$$\mathbf{A} \cdot \mathbf{x} = \mathbf{b} \tag{2.0.3}$$

Here, and throughout the book, we use a raised dot to denote matrix multiplication, *or* the multiplication of a matrix and a vector, *or* the dot product of two vectors.

This usage is nonstandard, but we think it adds clarity: the dot is, in all of these cases, a *contraction* operator that represents the sum over a pair of indices, for example

$$\begin{aligned}
 \mathbf{C} &= \mathbf{A} \cdot \mathbf{B} &\iff c_{ik} &= \sum_j a_{ij} b_{jk} \\
 \mathbf{b} &= \mathbf{A} \cdot \mathbf{x} &\iff b_i &= \sum_j a_{ij} x_j \\
 \mathbf{d} &= \mathbf{x} \cdot \mathbf{A} &\iff d_j &= \sum_i x_i a_{ij} \\
 q &= \mathbf{x} \cdot \mathbf{y} &\iff q &= \sum_i x_i y_i
 \end{aligned} \tag{2.0.4}$$

In matrices, by convention, the first index on an element a_{ij} denotes its row and the second index its column. For most purposes you don't need to know how a matrix is stored in a computer's physical memory; you just reference matrix elements by their two-dimensional addresses, e.g., $a_{34} = a[3][4]$. This C++ notation can in fact hide a variety of subtle and versatile physical storage schemes, see §1.4 and §1.5.

2.0.1 Nonsingular versus Singular Sets of Equations

You might be wondering why, above, and for the case $M = N$, we credited only a “good” chance of solving for the unknowns. Analytically, there can fail to be a solution (or a unique solution) if one or more of the M equations is a linear combination of the others, a condition called *row degeneracy*, or if all equations contain certain variables only in exactly the same linear combination, called *column degeneracy*. It turns out that, for square matrices, row degeneracy implies column degeneracy, and vice versa. A set of equations that is degenerate is called *singular*. We will consider singular matrices in some detail in §2.6.

Numerically, at least two additional things prevent us from getting a good solution:

- While not exact linear combinations of each other, some of the equations may be so close to linearly dependent that roundoff errors in the machine render them linearly dependent at some stage in the solution process. In this case your numerical procedure will fail, and it can tell you that it has failed.
- Accumulated roundoff errors in the solution process can swamp the true solution. This problem particularly emerges if N is too large. The numerical procedure does not fail algorithmically. However, it returns a set of x 's that are wrong, as can be discovered by direct substitution back into the original equations. The closer a set of equations is to being singular, the more likely this is to happen, since increasingly close cancellations will occur during the solution. In fact, the preceding item can be viewed as the special case in which the loss of significance is unfortunately total.

Much of the sophistication of well-written “linear equation-solving packages” is devoted to the detection and/or correction of these two pathologies. It is difficult to give any firm guidelines for when such sophistication is needed, since there is no such thing as a “typical” linear problem. But here is a rough idea: Linear sets with N no larger than 20 or 50 are routine if they are not close to singular; they rarely

require more than the most straightforward methods, even in only single (that is, float) precision. With double precision, this number can readily be extended to N as large as perhaps 1000, after which point the limiting factor anyway soon becomes machine time, not accuracy.

Even larger linear sets, N in the thousands or millions, can be solved when the coefficients are sparse (that is, mostly zero), by methods that take advantage of the sparseness. We discuss this further in §2.7.

Unfortunately, one seems just as often to encounter linear problems that, by their underlying nature, are close to singular. In this case, you *might* need to resort to sophisticated methods even for the case of $N = 10$ (though rarely for $N = 5$). Singular value decomposition (§2.6) is a technique that can sometimes turn singular problems into nonsingular ones, in which case additional sophistication becomes unnecessary.

2.0.2 Tasks of Computational Linear Algebra

There is much more to linear algebra than just solving a single set of equations with a single right-hand side. Here, we list the major topics treated in this chapter. (Chapter 11 continues the subject with discussion of eigenvalue/eigenvector problems.)

When $M = N$:

- Solution of the matrix equation $\mathbf{A} \cdot \mathbf{x} = \mathbf{b}$ for an unknown vector $\mathbf{x}$ (§2.1 – §2.10).
- Solution of more than one matrix equation $\mathbf{A} \cdot \mathbf{x}_j = \mathbf{b}_j$, for a set of vectors $\mathbf{x}_j$, $j = 0, 1, \dots$, each corresponding to a different, known right-hand side vector $\mathbf{b}_j$. In this task the key simplification is that the matrix $\mathbf{A}$ is held constant, while the right-hand sides, the $\mathbf{b}$'s, are changed (§2.1 – §2.10).
- Calculation of the matrix $\mathbf{A}^{-1}$ that is the matrix inverse of a square matrix $\mathbf{A}$, i.e., $\mathbf{A} \cdot \mathbf{A}^{-1} = \mathbf{A}^{-1} \cdot \mathbf{A} = \mathbf{1}$, where $\mathbf{1}$ is the identity matrix (all zeros except for ones on the diagonal). This task is equivalent, for an $N \times N$ matrix $\mathbf{A}$, to the previous task with N different $\mathbf{b}_j$'s ($j = 0, 1, \dots, N - 1$), namely the unit vectors ($\mathbf{b}_j =$ all zero elements except for 1 in the j th component). The corresponding $\mathbf{x}$'s are then the columns of the matrix inverse of $\mathbf{A}$ (§2.1 and §2.3).
- Calculation of the determinant of a square matrix $\mathbf{A}$ (§2.3).

If $M < N$, or if $M = N$ but the equations are degenerate, then there are effectively fewer equations than unknowns. In this case there can be either no solution, or else more than one solution vector $\mathbf{x}$. In the latter event, the solution space consists of a particular solution $\mathbf{x}_p$ added to any linear combination of (typically) $N - M$ vectors (which are said to be in the nullspace of the matrix $\mathbf{A}$). The task of finding the solution space of $\mathbf{A}$ involves

- Singular value decomposition of a matrix $\mathbf{A}$ (§2.6).

If there are more equations than unknowns, $M > N$, there is in general no solution vector $\mathbf{x}$ to equation (2.0.1), and the set of equations is said to be *overdetermined*. It happens frequently, however, that the best “compromise” solution is sought, the one that comes closest to satisfying all equations simultaneously. If closeness is defined in the least-squares sense, i.e., that the sum of the squares of the differences between the left- and right-hand sides of equation (2.0.1) be mini-

mized, then the overdetermined linear problem reduces to a (usually) solvable linear problem, called the

- Linear least-squares problem.

The reduced set of equations to be solved can be written as the $N \times N$ set of equations

$$(\mathbf{A}^T \cdot \mathbf{A}) \cdot \mathbf{x} = (\mathbf{A}^T \cdot \mathbf{b}) \quad (2.0.5)$$

where $\mathbf{A}^T$ denotes the transpose of the matrix $\mathbf{A}$. Equations (2.0.5) are called the *normal equations* of the linear least-squares problem. There is a close connection between singular value decomposition and the linear least-squares problem, and the latter is also discussed in §2.6. You should be warned that direct solution of the normal equations (2.0.5) is not generally the best way to find least-squares solutions.

Some other topics in this chapter include

- Iterative improvement of a solution (§2.5)
- Various special forms: symmetric positive-definite (§2.9), tridiagonal (§2.4), band-diagonal (§2.4), Toeplitz (§2.8), Vandermonde (§2.8), sparse (§2.7)
- Strassen's "fast matrix inversion" (§2.11).

2.0.3 Software for Linear Algebra

Going beyond what we can include in this book, several good software packages for linear algebra are available, though not always in C++. LAPACK, a successor to the venerable LINPACK, was developed at Argonne National Laboratories and deserves particular mention because it is published, documented, and available for free use. ScaLAPACK is a version available for parallel architectures. Packages available commercially include those in the IMSL and NAG libraries.

Sophisticated packages are designed with very large linear systems in mind. They therefore go to great effort to minimize not only the number of operations, but also the required storage. Routines for the various tasks are usually provided in several versions, corresponding to several possible simplifications in the form of the input coefficient matrix: symmetric, triangular, banded, positive-definite, etc. If you have a large matrix in one of these forms, you should certainly take advantage of the increased efficiency provided by these different routines, and not just use the form provided for general matrices.

There is also a great watershed dividing routines that are *direct* (i.e., execute in a predictable number of operations) from routines that are *iterative* (i.e., attempt to converge to the desired answer in however many steps are necessary). Iterative methods become preferable when the battle against loss of significance is in danger of being lost, either due to large N or because the problem is close to singular. We will treat iterative methods only incompletely in this book, in §2.7 and in Chapters 19 and 20. These methods are important but mostly beyond our scope. We will, however, discuss in detail a technique that is on the borderline between direct and iterative methods, namely the iterative improvement of a solution that has been obtained by direct methods (§2.5).

CITED REFERENCES AND FURTHER READING:

Golub, G.H., and Van Loan, C.F. 1996, *Matrix Computations*, 3rd ed. (Baltimore: Johns Hopkins University Press).